מבוא למדעי המחשב (67101- 2) - סמסטר ב', מבחן מועד ב' 15.8.21

מרצה הקורס: ד"ר אריה שלזינגר

במבחן זה יש לענות על כל 10 השאלות בחלק א׳, ולבחור 4 מתוך 6 השאלות בחלק ב׳ (אין לענות על יותר מ-4 שאלות בחלק ב׳). יש לענות על כל השאלות בגוף השאלון - בתוך ״קופסאות התשובה״ בלבד. אין להגיש דפי טיוטה - הם לא יבדקו. את התשובות יש לכתוב בעט שחור או כחול בלבד. אין לכתוב בשוליים.

חומר מותר לשימוש במבחן: אין. משך הבחינה : 3 שעות.

חלק א׳ - שאלות סגורות (40% - יש לענות על כל 10 הסעיפים, כל סעיף מזכה ב - 4 נק׳)

בחלק זה של המבחן יש לכתוב את התשובות הסופיות בלבד בתוך המשבצת המתאימה. אין צורך לנמק.

שאלה	תשובה
1	
2	
3	
4	
5	

שאלה	תשובה
6	
7	
8	
9	
10	

סמנו כאן על איזה שאלות בחרתם לענות **בחלק ב׳**, הקיפו בעיגול

1	**2**	**3**	**4**	**5**	**6**

חלק א׳

1. (4%) מה ידפיס קטע הקוד הבא:

```
def f1(x, y):
    if x == y:
        return True
    if x > y:
        return False
    if x % 10 == y % 10:
        return f1(x//10, y//10)
    else:
        return f1(x, y//10)

print(list(map(f1,[23],(4253,))))
```

---

2. (4%) מה ידפיס קטע הקוד הבא:

```
k = lambda k, p: lambda: k - p
t = (lambda a, b: a(10, b)()) (k, 2)
print(t)
```

---

3. (4%) מה היא סיבוכיות זמן הריצה של הפונקציה **f3** כפונקציה של **n**?

```
def f3(n):
    a=0
    i, j = n, 1
    while i>0:
        i//=2
        while j<n:
            j*=2
            for k in range(n):
                k+=2
                a+=i+j+k
    return(a)
```

---

4. (4%) מה ידפיס קטע הקוד הבא:

```
def f4(x):
    def h(x):
        return x*g(x-1,h)
    def g(x,g):
        if x <= 0:
            return 1
        return g(x)
    return g(x,h)

print(f4(2))
```

5. (4%) מה היא סיבוכיות זמן הריצה של הפונקציה **f5** כפונקציה של n?

```
def f5(n):
    b, k, c, d = 0, 1, 2, n + 1
    while k <= n:
        if (len([t for t in range(c, d) if k % t == 0])) > 1:
            b +=  1
        k += 1
    return b
```

6. (4%) מה ידפיס קטע הקוד הבא:

```
liz = [[y * 3 for y in range(x)] for x in range(5)][::-1]
print(liz[:2])
```

7. (4%) מה ידפיס קטע הקוד הבא:

```
f7 = lambda f, g: g(f)
print(f7(lambda x: x + 7, lambda x: x)(-5) )
```

8. (4%) מה ידפיס קטע הקוד הבא:

```
def dec(other):
    def dec2():
        return other(6, 10)
    print(dec2())
    return print

@dec
def func(a, b):
    return a - b

func(6, 10)
```

9. (4%) מה ידפיס קטע הקוד הבא:

```
def f9():
    try:
        f20()
        print("from f9")
    except ValueError:
        print("Great Value")

def f20():
    raise ValueError

try:
    f9()
    print("from globe")
except Exception:
    print("Who goes there ?")

print("out")
```

```
class Node:
    def __init__(self, data = None, next = None):
        self.data = data
        self.next = next

    def __repr__(self):
        return 'Node(' +repr(self.data) + ( ',' + repr(self.next)) if self.next else '' + ')'

    def __str__(self):
        st = '<'
        while self.next is not None:
            st+=str(self.data)+' '
            self=self.next
        return st+str(self.data)+'>'
```

10. (4%) מה ידפיס קטע הקוד הבא:

הניחו ש-**lnk** היא רשימה מקושרת וה-**Node** מוגדר לעיל.

```
def f10(lnk):
    if lnk is not None:
        yield from f10(lnk.next)
        yield lnk.data

print(next(f10(Node(1, Node(3)))))
print(list(f10(Node(1, Node(3, Node(2))))))
```

חלק ב' – שאלות תכנות

(60% - יש לענות רק על **4 מתוך 6** השאלות, לכל שאלה 15 נק')

– יש לענות על כל שאלה רק בתוך קופסת הטקסט שניתנה בדף.

– מומלץ להשתמש במחברת הטיוטה כדי לגבש את הפתרון ורק כאשר הוא מושלם להעתיקו לטופס הבחינה.

– אין חובה לכתוב הערות או תיעוד.

– יש לכתוב קוד ברור ויעיל ככל הניתן (מבחינת זמן ריצה וזיכרון) על דפי התשובה של השאלה. אם יש צורך, ישנם דפים נוספים בסוף המבחן.

– אם לא נאמר מפורשות אחרת, ניתן להניח כי כל הקלטים לפונקציות הינם תקינים.

חלק ב׳

1. (15%) לפניכם המחלקה **TreeNode** שמייצגת קודקוד בעץ חיפוש בינארי:

```
class TreeNode:
      def __init__(self, value, left=None, right=None):
            self.value = value
            self.left = left
            self.right = right
```

נזכיר שעץ חיפוש בינארי מקיים שעבור כל קודקוד בעץ, כל הערכים בתת העץ השמאלי קטנים יותר מהערך שבקודקוד, וכל הערכים בתת העץ הימני גדולים יותר מהערך שבקודקוד.

כתבו פונקציה **kth_smallest(root, k)** אשר מקבלת:

- **root** - שורש של עץ חיפוש בינארי (אובייקט מסוג **TreeNode**)
- **k** - מספר טבעי

הפונקציה תחזיר את הערך ה-**k**-י הקטן בגודלו בעץ.
למשל:

```
root = TreeNode(8)
root.left = TreeNode(5)
root.right = TreeNode(14)
root.left.left = TreeNode(0)
root.left.right = TreeNode(6)
root.right.right = TreeNode(27)
root.right.right.right = TreeNode(300)
```

מגדיר את העץ:

```
          8
       /     \
      5      14
    /  \       \
   0    6       27
                 \
                 300
```

```
print(kth_smallest(root, 1)) -> 0 (הערך ה-1 הכי קטן)
print(kth_smallest(root, 2)) -> 5 (הערך ה-2 הכי קטן)
print(kth_smallest(root, 3)) -> 6 (הערך ה-3 הכי קטן)
print(kth_smallest(root, 7)) -> 300 (הערך ה-7 הכי קטן)
```

ניתן להניח שהעץ הבינארי תקין, וכל ערך מופיע לכל היותר פעם אחת בעץ. ניתן להניח שיקראו לפונקציה עם ערך **k** שקטן או שווה לכמות הקודקודים בעץ. יש לכתוב פתרון יעיל ככל הניתן (מבחינת זמן ריצה וזיכרון).

```
def kth_smallest(root, k):
```


2. (15%) האיבר הראשון של סדרת **Collatz** הוא מספר שלם חיובי **k**. אם **k** הוא זוגי, האיבר הבא בסדרה יהיה **k/2**. אחרת האיבר הבא יהיה **3k+1**. את יתר איברי הסדרה בונים באותה דרך, בהתאם לאיבר הקודם, וזו מסתיימת כאשר מגיעים לאיבר שערכו **1**,(הניחו שהסדרה תמיד מגיעה ל **1**). דוגמאות:

```
4, 2, 1
5, 16, 8, 4, 2, 1
13, 40, 20, 10, 5, 16, 8, 4, 2, 1
```

כתבו את הפונקציה **map_collatz(k, fun)** אשר מקבלת מספר שלם חיובי **k** ופונקציה בעלת ארגומנט אחד **fun(x)**. הפונקציה **map_collatz** תפעיל את **fun** על כל אחד מאברי סדרת **Collatz** (המתחילה ב-**k** שהתקבל). ותחזיר את אורך הסדרה. אפשר להיעזר בפונקציות עזר. דוגמאות:

```
def f1(k):
    print((k+1)/0.25)
>> ans = map_collatz(6, f1)
28.0
16.0
44.0
24.0
68.0
36.0
20.0
12.0
8.0
>> print(ans)
9
```

```
>> ans = map_collatz(5, print)
5
16
8
4
2
1
>> print(ans)
6
```

```
liz = []
l_len = map_collatz(14, liz.append)
print(liz, l_len)
[13, 40, 20, 10, 5, 16, 8, 4, 2, 1] 10
```

```
zed = set()
z_len = map_collatz(10, zed.add)
print(zed, z_len)
{1, 2, 4, 5, 8, 10, 16} 7
```

```
def map_collatz(k, fun):
```

3. (15%) כתבו פונקציה **split_sorted(head)** אשר מקבלת **head** – מצביע לראש של רשימה מקושרת מטיפוס **Node** , ומפצלת את הרשימה לתת רשימות ממוינות. אין לייצר קודקודים חדשים. הפונקציה תחזיר רשימה (**list**) של מצביעים לראשי הרשימות הממוינות. הפונקציה לא משנה את סדר הקודקודים אלא רק מפרידה אותם בהתאם לסדר שבו הם נמצאים, כך שהרשימות המופרדות הן ממוינות.

הניחו שזו המחלקה Node (הפונקציה repr משמשת להצגה ברורה של הדוגמא):

```
class Node:
      def __init__(self, data, next=None):
          self.data = data
          self.next = next
      def __repr__(self):
          return repr(self.data) + "->" + repr(self.next)
```

דוגמאות:

```
my_node = Node(1, Node(2, Node(3, Node(2, Node(1,Node(10))))))
split_sorted(my_node)
> [1->2->3->None, 2->None, 1->10->None]

my_node2 = Node(1, Node(2, Node(3, Node(4))))
split_sorted(my_node2)
> [1->2->3->4->None]

my_node3 = Node(10, Node(9, Node(8, Node(3, Node(1)))))
split_sorted(my_node3)
> [10->None, 9->None, 8->None, 3->None, 1->None]
```

```
def split_sorted(head):
```

4. (15%) כתבו פונקציה **רקורסיבית** `guess_pass(pwd, text)` אשר מקבלת שתי מחרוזות: **pwd,** **text**. אם כל התווים של **pwd** נמצאים לפי הסדר בתוך המחרוזת **text**, הפונקציה מחזירה רשימה של האינדקסים של המופע הראשון של כל אחד מהתווים בתוך ה-text.
אם התווים של **pwd** או חלקם לא נמצאים ב-**text**, או נמצאים אך לא לפי הסדר שלהם ב-**pwd**, הפונקציה תחזיר **False**. אם **text** היא מחרוזת ריקה, הפונקציה תחזיר **False**, ואם המחרוזת **pwd** ריקה, הפונקציה תחזיר רשימה ריקה. ה-**text** יכול להכיל כפילויות. דוגמאות:

>pwd = "abc-123" **>text** = "acb-123" **>guess_pass(pwd, text)** **False**	**>**pwd = "abc-123" **>**text= "**a**1**b**2**c**34/-**1**x**2**gh**3**n" >guess_pass(pwd, text) [0, 2, 4, 8, 9, 11, 14]	>pwd="abc-123" **>**text="**abc-123**abc-123" >guess_pass(pwd, text) [0, 1, 2, 3, 4, 5, 6]
>pwd = "" **>text = ""** **>guess_pass(pwd, text)** **[]**	**>pwd = "a"** **>text = ""** **>guess_pass(pwd, text)** **False**	**>pwd = ""** **>text = "abcd"** **>guess_pass(pwd, text)** **[]**

```
def guess_pass(pwd, text):
```

5. (15%) כתבו פונקציה `n_of_sums(n,k,fun)` אשר מקבלת שני מספרים שלמים וחיובים **`n>=k`**, ופונקציה **`fun`**, בעלת ארגומנט אחד חיובי, אשר מקיימת **`fun(x)<x`**.
הפונקציה **`n_of_sums`** תחזיר את **מספר הדרכים השונות** לחלוקת **n** לשלמים חיוביים, שנמצאים בין איברי הסדרה (היורדת) הבאה: **`k,fun(k),fun(fun(k)),...`**.
מספר יופיע בסכום מסויים רק פעם אחת. דוגמאות:

עם **`lambda a: a-1`**, מתקבלת הסדרה: **`4,3,2,1`** וממנה ניתן להרכיב שני סכומים שונים ששווים **6: 3+2+1 , 4+2**	`>n_of_sums(6, 4, lambda a: a-1)` `2 # (4+2, 3+2+1)`
עם **`lambda b: b-2`**, מתקבלת הסדרה: **`12,10,8,6,4,2`** וממנה ניתן להרכיב ארבעה סכומים שונים ששווים **12:** **6+4+2, 8+4, 10+2, 12**	`>n_of_sums(12, 12, lambda b:b-2 )` `4 # (12, 10+2, 8+4, 6+4+2 )`
עם **`lambda x: x//2`** , מתקבלת הסדרה: **`8,4,2,1`** וממנה ניתן להרכיב סכום אחד ששווה **11: 8+2+1**	`>n_of_sums(11, 8,lambda x: x//2 )` `1 # (8+2+1)`
עם **`lambda y: y-3`**, מתקבלת הסדרה: **`4,1`** וממנה לא ניתן להרכיב אף סכום ששווה ל 7.	`>n_of_sums (7 , 4, lambda y: y-3)` `0`

`def n_of_sums(n,k,fun):`

6. מרווח (`Interval`) הוא קטע רציף על הישר הממשי. אם הקצוות שלו, `a,b`, סופיים ומוכלים בו, המרווח נקרא סגור; סימון: `[a,b]` . המרווח `[a,a]` מכיל רק את `a`.

ניתן להגדיר פעולות אריתמטיות בין מרווחים, והתוצאה היא גם מרווח. למשל תוצאת החיבור בין שני מרווחים: `[f,g] = [a,b]+[c,d]`, כאשר : `f=a+c, g=b+d`

נסמן פעולות אריתמטיות בין שני מרווחים כך: `[t,k] = [a,b] op [c,d]`,
תוצאת הפעולה היא המרווח `[t,k]`, ו-`op` היא אחת מבין הפעולות : `+, -, *, /`.

המרווח `[t,k]` הוא אוסף הערכים שמתקבלים מכל הפעולות האפשריות: `x op y`
כאשר: `x` מתוך `[a,b]`, ו-`y` מתוך `[c,d]`. גבולות המרווח החדש יהיו:

```
t = min{a op c, a op d, b op c, b op d}
k = max{a op c, a op d, b op c, b op d}
```

(הגדרת החילוק לעיל ״נכונה״ רק אם המרווח `[c,d]` אינו מכיל את ה-`0`).
אין קשר בין סימני הסוגריים בשורות לעיל לבין הסימני הסוגריים שבשימוש ב **Python** .

כתבו את `class Interval`, שהאובייקטים שלה מייצגים מרווחים סגורים בין שני מספרים.
במחלקה `Interval`, מלבד השיטה `__init__` , אשר מקבלת גבולות של מרווח, ומייצרת אובייקט שמייצג אותו, ואחת מבין השיטות `__str__` , או `__repr__` , להדפסת האובייקט, יהיו גם השיטות לאריתמטיקה של מרווחים שתוארו לעיל , עבור הפעולות : `+,-,*,/` .
את אלה תממשו באמצעות השיטות המיוחדות :
`__add__, __sub__, __mul__, __truediv__`. הניחו שהקלט תקין. דוגמאות:

`inty=Interval(2,4)`
`inty1=Interval(3,1)`
`inty2=Interval(-1,2)`
`inty3=Interval(4,4)`
`inty4=Interval(0,0)`	

`print(inty/inty2)`	**ValueError: [-1, 2] contains 0, so it can not be used**
`print(inty/(inty3+inty2))`	`[0.333333, 1.33333]`
`print(inty3-inty2)`	`[2, 5]`
`print(inty*inty3)`	`[8,16]`
`print(inty*inty3-inty2)`	`[6, 17]`
`print(inty*(inty3-inty2))`	`[4, 20]`

```
class Interval:
```

דפי כתיבה נוספים:

בהצלחה